A Lexically Filtered Meron-Cluster Benchmark: Exact Ground Truth, Mechanized Cancellation Lemmas, and a Pilot That Exposes Answer Leakage

Jeremy H. Carroll

2026-10-03 · Version 0.1.0 · Creative Commons Attribution 4.0 International (CC-BY-4.0)

Abstract

This paper packages one small, exactly solvable instance of the meron-cluster mechanism of Chandrasekharan and Wiese and reports a pilot language-model run on it that cannot support a discovery claim. The instance is a four-site periodic chain with 256 breakup graphs and 1,296 configurations, of which 192 are negative. An exact oracle built on SQLite and rational arithmetic finds that the 57 graphs containing a meron cluster cancel exactly and that the other 199 give the signed sum $Z = 912$; an independent transfer-matrix trace also gives 912. The per-orbit cancellation lemmas are checked in Lean 4. A substring filter and five prompt builders pose the instance as a graph problem without twelve domain words. In the retained pilot (five models accessed through OpenRouter, one run per model and arm), no unassisted prompt produced a passing answer: 0 of 9 cells. Every pass came after the prompt contained the answer. The scorer's feedback states "Z must equal 912" and the split into 57 cancelling and 199 surviving subgraphs, and the fifth arm also dictates the sign formula and the cancellation step. Even the unassisted prompts state the configuration count and the average sign, from which Z follows by one multiplication. The pilot therefore measures instruction following under answer-bearing prompts. We state the design changes a valid rerun needs; that rerun requires authorized live API calls and has not been made. Section 6.2 records one further exact run of the same SQLite construction at six sites.

1. Introduction

Quantum Monte Carlo sampling of fermions is limited by the sign problem. Configuration weights can be negative, the average sign of the sampled weights usually decays exponentially with system size and inverse temperature, and the statistical error of a signed estimate grows accordingly. Troyer and Wiese showed that a generic solution of the sign problem for a class of frustrated systems would solve NP-hard problems [2]; this is a statement about the worst case. For particular models the sign problem has nevertheless been solved. In the meron-cluster method of Chandrasekharan and Wiese [1], each configuration belongs to a breakup graph whose clusters can be flipped independently. Clusters whose flip reverses the sign, called merons, make the whole orbit of a graph cancel, and the remaining orbits contribute positive weight. Later work solved further classes, for example half-filled spin-polarized models on bipartite lattices with fermion bags [3].

This paper contains:

1. the finite cancellation argument (Section 2), with per-orbit lemmas checked in Lean 4 (Section 7);
2. an exact instance with two independent exact computations of $Z = 912$ (Section 3);
3. a substring filter and five prompt builders that pose the instance as a graph problem (Section 4);

4. a pilot run with five language models accessed through OpenRouter, one retained output from a sixth, anonymous model, and an analysis showing that every passing run received the answer in its prompt (Sections 5 and 6);
5. concrete physical implementations and exact rational cancellation on higher-dimensional lattices: a two-dimensional staggered Gross-Neveu meron cluster solver on T^2 (Section 6.5), a three-dimensional closed meron string loop linking and winding cancellation construction on T^3 (Section 6.6), and an automated multi-turn model ladder benchmark on OpenRouter evaluating Γ_{38} wire emission (Section 6.7).

It does not show that any model discovered the mechanism.

2. Meron-cluster cancellation on a finite orbit

Let g be a breakup graph with $k = k(g)$ clusters $C_1, \dots, C_k$. The configurations compatible with g are obtained by flipping clusters independently; flipping one cluster changes no other cluster, so they are indexed by orientation vectors $s \in \{-1, +1\}^k$, with $s = (1, \dots, 1)$ the reference orientation. Write $w(g, s)$ for the signed weight of a configuration and $\operatorname{sgn}(g, s)$ for its sign. Two properties of the model are assumed.

Assumption 2.1 (equal weights on an orbit). For each g there is $W_0(g) > 0$ with $|w(g, s)| = W_0(g)$ for every s.

This is a property of the model's cluster rules: the breakup weights are chosen so that the weight of a configuration depends only on its breakup graph. It does not follow from the clusters being disjoint. In the fixture of Section 3 the construction gives every configuration the weight of its graph, and the oracle confirms that no configuration differs.

Assumption 2.2 (character law). There are indicators $I_1, \dots, I_k \in \{0, 1\}$ with

$$\operatorname{sgn}(g, s) = \prod_{j=1}^{k} s_j^{I_j}.$$

A cluster with $I_j = 1$ is a meron: flipping it alone reverses the sign. The prompts call such cycles "odd-winding"; this paper uses only the operational definition. Chandrasekharan and Wiese establish the character law for their models [1]. In the fixture the oracle checks it against the fermionic exchange sign of every one of the 1,296 configurations.

Lemma 2.3 (orbit factorization). Under Assumptions 2.1 and 2.2,

$$\sum_{s \in \{-1,+1\}^k} w(g, s) = W_0(g) \prod_{j=1}^{k} \Big(\sum_{s_j = \pm 1} s_j^{I_j} \Big).$$

Proof. By the assumptions, $w(g, s) = W_0(g) \prod_j s_j^{I_j}$. The sum runs over the full product set $\{-1, +1\}^k$ and the summand is a product of one factor per coordinate, so by distributivity the sum equals the product of the coordinate sums. □

Lemma 2.4 (cancellation, Claim MSP-01). If $I_i = 1$ for some i, the orbit sum of g is zero.

Proof. The factor for coordinate i is $(+1) + (-1) = 0$. Equivalently, flipping cluster i is a weight-preserving, sign-reversing involution without fixed points (Claim MSP-04), which pairs the configurations of g into cancelling pairs. □

Lemma 2.5 (positivity, Claim MSP-02). If $I_j = 0$ for every j, every configuration of g has sign $+1$ and the orbit sum is $2^k W_0(g) > 0$.

Proof. Each factor is $1 + 1 = 2$. □

Theorem 2.6 (reduction, Claim MSP-03). Let $m_g = \sum_j I_j$ be the number of meron clusters of g. Then

$$Z = \sum_g \sum_s w(g, s) = \sum_{g\,:\,m_g=0} 2^{k(g)} W_0(g) \;\geq\; 0.$$

Proof. Sum Lemma 2.4 over the graphs with $m_g > 0$ and Lemma 2.5 over the graphs with $m_g = 0$. □

Section 7 lists which of these statements are checked in Lean 4. The per-orbit identities are checked with unit weight; the factor $W_0(g)$ and the sum over graphs are elementary and are not mechanized.

3. The exact fixture and its oracle

The fixture is built by the function `meron_database` in `code/meron_exact_model.py`, a copy of the repository's exact model (Section 9), with four sites, two repetitions and $q = 3$. It is a periodic chain of four sites whose checkerboard split has four time slices and eight two-site interaction plaquettes. Each plaquette takes one of two breakups, so there are $2^8 = 256$ breakup graphs. The two breakup weights are 1 and $(q-1)/2$ with $q = e^{\varepsilon t}$; at $q = 3$ both equal 1, so every graph and every configuration has weight 1. For each graph, SQLite builds its clusters, the 2^k orientations, the occupation states they induce, and the fermionic exchange sign of each configuration, using exact rational arithmetic on reduced fraction strings.

Table 1. Ground truth of the fixture (tests in `code/tests/test_meron_cluster_solution.py`).

Quantity	Value
Breakup graphs	256
Configurations	1,296
Negative configurations	192
Graphs with a meron cluster (orbit sum 0)	57
Graphs without a meron cluster	199
Signed sum Z	912
Absolute sum Z_{abs}	1,296
Average sign Z/Z_{abs}	19/27
Transfer-matrix trace, computed without clusters	912

Because all weights are 1, $Z = 1{,}104 - 192 = 912$. The 57 cancelling graphs hold 384 configurations, 192 of them negative, and each of these orbits sums to zero; the 199 other graphs hold the remaining 912 configurations, all with sign $+1$, as Theorem 2.6 requires (Claim MSP-05). The oracle also confirms Assumptions 2.1 and 2.2 on the fixture (Claim MSP-07): no configuration's weight differs

from its graph weight, and every sign equals the product of the single-flip signs of the flipped clusters. An independent computation, `meron_transfer_trace`, multiplies two-site transfer factors in the occupation basis (diagonal entries 1 and $(q+1)/2$; hopping entries $(q-1)/2$ with the fermionic exchange sign) and takes the trace. It uses no clusters and also gives 912 (Claim MSP-06).

4. The lexical filter and the five prompts

`assert_lexical_hygiene` in `code/blind_problem.py` rejects any text that contains, ignoring case, one of twelve substrings: meron, fermion, physics, hamiltonian, hubbard, chandrasekharan, wiese, trotter, fock, anticommut, quantum monte carlo and qmc. Every prompt builder applies it, and all five prompt builders pass the filter (Claim MSP-08). It is a substring filter only. The prompts keep sign-problem vocabulary that is not on the list, such as "Boltzmann-like weight", "partition value Z", "average sign" and "sign cancellation", and they state quantities that fix the answer (Section 5.3).

The builders in `code/prompt_builder.py` produce five prompts:

- Arm 1: the problem text (a graph with 256 decomposition subgraphs, the configuration counts, and a list of objectives), four sample subgraphs with the sign and weight of every configuration, and a JSON output instruction. It has 4,320 characters.
- Arm 2: the counts and, for the same four samples, only the number of cycles, their lengths, the number of flip states, the number of odd-winding cycles and the weight, with instructions to prove the cancellation property, count the subgraphs without odd-winding cycles, and compute Z.
- Arm 3: the same sample data in a compact notation: $\Sigma = [C_0{:}4][C_1{:}4]\ldots$ for the cycle lengths, $\dim(H_1) = k$, $\Lambda_{\text{flips}} = 2^k$, I_{odd} for the number of odd-winding cycles and W_{base} for the weight. The prompt names an eight-letter alphabet $\{E, R, C, I, \Lambda, D, M, T\}$, but the listed data use only these fields. It omits the problem narrative and the configuration lists, which makes it 1,124 characters long, 74% shorter than Arm 1 (4,320 characters; Claim MSP-14). It carries less information; it is not the same information in fewer characters.
- Arm 4: the Arm 1 problem text and samples over up to five rounds, with the scorer's correction strings appended after a failed round (Section 5.4).
- Arm 5: the Arm 3 notation, the correction strings collected in Arm 4, and a four-step outline of the argument (Section 5.4).

The four sample subgraphs are the first four by identifier. Each has two to four cycles, weight 1 and no odd-winding cycle. Every arm asks for the same JSON object; Section 5.1 lists the fields that are scored.

5. The pilot run

5.1 Design as run

The harness `code/run_blind_benchmark.py`, as committed with the results in commit e2a1e9fd0, queried five language models accessed through OpenRouter: Llama 3.3 70B (`meta-llama/llama-3.3-70b-instruct`), DeepSeek V4 Flash (`deepseek/deepseek-v4-flash-0731`), GLM 5.3 Flash (`z-ai/glm-5.3-flash`), Hy4 Preview (`tencent/hy4-preview`) and Qwen 3.7 Max (`qwen/qwen3.7-max`). Arms 1 to 3 used the first three models; Arm 4 used the same three for up to five rounds; Arm 5 used Llama 3.3 70B, Hy4 Preview and Qwen 3.7 Max, with the correction strings collected from the Arm 4 runs. Each model and arm ran once ($n = 1$), with no repeated trials. The harness code

sets temperature 0.2, a limit of 3,000 completion tokens and a strict JSON-schema response format, and sets no seed. The report does not record the parameters actually sent or which provider served each call. When a response had no answer text, the harness scored the provider's reasoning text instead.

The report `code/results/master_evaluation_report.json` (timestamp 2026-09-28 13:57:53 UTC) lists 22 calls across five models, one run per model and arm, with the scores of Table 2 (Claim MSP-09). `code/results/receipt.json` records the SHA-256 digest of the report, which matches the retained bytes; its `status` field is a constant written by the harness, not the result of a check. The harness estimated the spend at 0.01496 USD from its configured per-token prices; this is not a billed amount (Claim MSP-09).

Each answer was scored against the oracle on four items: (1) `exact_partition_z` equals 912; (2) `cycle_independence` is truthy (any non-empty value counts); (3) `cycle_parity_invariant_found` is truthy and the `cancellation_mechanism` text contains one of eight keywords, such as "canc", "flip" or "zero"; (4) `zero_odd_sector_subgraphs` equals 199. A cell passes with 4 of 4. The rubric checks field values and keywords, not the validity of a derivation, and no second checker reviewed the answers. The scorer used for the run also passed item 4 when Z was 912 and the mechanism text contained the word "zero"; the current code drops that alternative and accepts integer spellings such as "912". Re-scoring the twelve retained single-turn answers with the current code reproduces every recorded score (Claim MSP-10).

5.2 Results

Table 2. Retained pilot results (`code/results/`). One run per model and arm. For Arm 4 the scores are listed by round.

Arm	Model	Rounds	Score (of 4)	Passed	Answer-bearing input? (yes/no)
1	Llama 3.3 70B	1	2	no	no
1	DeepSeek V4 Flash	1	0	no	no
1	GLM 5.3 Flash	1	0	no	no
2	Llama 3.3 70B	1	1	no	no
2	DeepSeek V4 Flash	1	0	no	no
2	GLM 5.3 Flash	1	0	no	no
3	Llama 3.3 70B	1	0	no	no
3	DeepSeek V4 Flash	1	0	no	no
3	GLM 5.3 Flash	1	0	no	no
4	Llama 3.3 70B	2	2, 4	yes (round 2)	yes (round 2)
4	DeepSeek V4 Flash	5	0, 0, 3, 0, 0	no	yes (rounds 2 to 5)
4	GLM 5.3 Flash	3	0, 0, 4	yes (round 3)	yes (rounds 2 and 3)
5	Llama 3.3 70B	1	3	no	yes
5	Hy4 Preview	1	0	no	yes
5	Qwen 3.7 Max	1	4	yes	yes
5	space-bunny-alpha	1	4 (re-scored)	yes	yes (inferred)

"No" in the last column means that the target values, $Z = 912$ and the count 199, were not stated. Every prompt, including those of Arms 1 to 3, states quantities from which $Z = 912$ follows by

arithmetic (Section 5.3). For the anonymous model the prompt was not retained, and "yes" is inferred from its output (Section 5.5).

5.3 The unassisted arms

No cell in Arms 1 to 3 passed: 0 of 9 (Claim MSP-11). Llama 3.3 70B gave $Z = 128$ in Arm 1 and $Z = 1{,}104$ in Arm 2, and returned an empty JSON object in Arm 3 after 3,000 completion tokens. DeepSeek V4 Flash returned no parseable answer in any of the three arms; the scored text was reasoning-style text that stopped at the 3,000-token limit. GLM 5.3 Flash returned an empty answer in Arms 1 and 3. In Arm 2 it returned a JSON object whose keys lie outside the schema: the key `Z` with value 912, a correct pairing argument under the key `cancellation_proof` (pair each configuration with the one obtained by flipping an odd-winding loop, which has the opposite sign), and the key `zero_odd_winding_subgraphs` with value 4. The scorer reads only the schema keys, so the cell scored 0 (Claim MSP-12); the count 4 is also wrong.

These prompts are less blind than their label suggests. Each states the number of configurations (1,296) and the average sign (19/27); Arms 1 and 2 also state the absolute sum (1,296), and Arm 1 states the numbers of positive and negative configurations (1,104 and 192). Since the average sign is Z/Z_{abs}, $Z = (19/27) \cdot 1{,}296 = 912$ follows by one multiplication. Each prompt also states the property to be proved: a subgraph with at least one sign-reversing cycle sums to zero. The count 199 is stated nowhere and cannot be computed from the prompt, because the four listed subgraphs all have no odd-winding cycle (Claim MSP-11). Item 4 could therefore be met only by guessing 199 or through the run-time alternative ($Z = 912$ and the word "zero").

5.4 The answer-bearing arms

In Arm 4, after each failed round the scorer appended one correction string for each failed item. Two of the four strings state the target:

> "The exact integer partition sum Z must equal 912. No approximation or empirical sample average is permitted."

> "All 57 subgraphs with odd-winding cycles sum to zero. The non-negative partition is computed strictly by summing the remaining 199 zero-odd-charge subgraphs."

The other two state the method: the sign factorizes over independent cycles, and "the sum factors as (1 + (-1)) * prod(...) = 0". Thus Arm 4 feedback states the target $Z = 912$ and the 57/199 split, while Arm 5 also dictates the sign formula and cancellation step (Claim MSP-13). Llama 3.3 70B scored 2 in round 1 ($Z = 128$), received the two strings above, and scored 4 in round 2. GLM 5.3 Flash scored 0 in rounds 1 and 2, with all four strings present from round 2 on, and 4 in round 3. DeepSeek V4 Flash did not pass in five rounds. The report keeps the scores and feedback of the Arm 4 rounds but not the answers, so it does not show whether the passing answers reported 199: under the run-time scorer, item 4 also passed on $Z = 912$ and the word "zero".

Arm 5 appended all four correction strings and a four-step outline that dictates the argument:

> "Step 2: Factorize the sign sigma(C) into the product of independent cycle signs: sigma(C) = prod_i s_i^{I(c_i)}."

> "Step 3: Show that when any I(c_i) = 1, the sum factors as (1 + (-1)) * prod_{j != i} (...) = 0."

> "Step 4: Count the subgraphs where all I(c_i) = 0 and compute Z."

The same prompt states 1,296 states and $\langle\sigma\rangle = 19/27$, and gives the odd-winding count (zero) of each of the four sample subgraphs. Qwen 3.7 Max scored 4; its scratchpad follows the four steps and uses the counts 57, 199 and $384 = 2 \cdot 192$. Llama 3.3 70B gave $Z = 912$ but entered 57 as the count of surviving subgraphs while its scratchpad states 199, and scored 3. Hy4 Preview returned no parseable answer within 3,000 tokens and scored 0. These passes measure instruction following and output formatting under prompts that contain the answer, not discovery.

5.5 The anonymous model

The sixth model is an unidentified model offered anonymously on OpenRouter under the identifier `stealth/space-bunny-alpha` in September 2026; its provider, size, and training are unknown and it may no longer be available, so these results cannot be reproduced. One output is retained, in `code/results/space_bunny_arm5_result.json`. The file holds the answer text and the provider's usage record only; it names no model and records no prompt, request parameters, latency or score (Claim MSP-15). The probe script committed with it (commit 0ad428e36) could not have produced it: as committed, building its Arm 5 prompt raises a KeyError, its scoring step calls a scorer method that does not exist, and it writes no file. The retained answer uses the counts 57 and 199. These appear in the probe's correction strings; they are not stated elsewhere in the Arm 5 prompt, and 199 cannot be computed from it. The prompt is therefore inferred to have been answer-bearing. Re-scoring the answer with the package scorer gives 4 of 4, although the answer does not conform to the output schema and its counts 57 and 199 come only from correction strings (Claim MSP-15): `cycle_independence` is a sentence rather than a boolean and `scratchpad` is a list rather than a string, and the scorer's coercion accepts both. An Arm 1 request to the same model was reported in version 0.3.0 as failing; its output was not retained, and it is not counted here.

6. What the pilot shows, and the design fix

The pilot shows three defects of the benchmark; it shows nothing about the models. First, the task text leaks: every prompt fixes Z through stated quantities and states the property to be proved. Second, the feedback leaks: the correction strings state $Z = 912$ and the 57/199 split, and Arm 5 adds an outline of the argument. Third, the scorer is weak: it checks values and keywords, and at run time it could pass item 4 without the count. Claims that language models discovered the cancellation mechanism or that prompt formats enabled discovery are withdrawn (Claim MSP-17). Similarly, claims that prompt compression reduced latency to 1.6 s are withdrawn (Claim MSP-18). With one run per cell, unrecorded decoding parameters and no pre-specified analysis, no comparison between arms or models is possible, and none is made.

A rerun that could test discovery needs:

1. task text that states neither the target nor quantities that fix it (such as the absolute sum with the average sign, or the sign counts), does not state the property to be proved, and supplies the data needed to compute every scored quantity, for example all 256 subgraphs;
2. feedback that names only which check failed, such as "check 1 failed", and never a target value or a method;
3. several trials per model and arm, with the decoding parameters fixed in advance and recorded for every request (temperature, token limit, seed where the provider supports one, and serving provider), and with every prompt and raw output retained;
4. a pre-specified analysis: the pass criterion, the primary comparison and the number of trials, fixed before the run;

5. a scorer that checks the reported count against the enumeration and does not award credit for keywords.

Such a rerun makes paid calls to external providers. It has not been run.

6.1 A separate finite witness task

The companion protocol in `docs/witness_benchmark.md` supplies complete raw edge and orientation-sign data for four selected orbits at $q = 5/2$ (Claim MSP-19). The public case labels do not encode their answers. Neither computed totals nor scorer correction strings are included. The requested answer is a component partition, every sign-reversing flip coordinate, a complete disjoint equal-weight opposite-sign pairing when it exists, and an exact rational total. The scorer checks the reported objects and each pair; keywords and truthy flags receive no credit.

This is a finite witness-recovery task with fully supplied inputs. Its host positive control is an offline check with zero provider calls, not a model trial (Claim MSP-20). The only committed `results.json` is `docs/witness_host_control_2026_10_02/results.json`, with zero provider calls and null usage; unverified live-trial claims are formally withdrawn (Claim MSP-22). Any live use records the full prompt, request parameters, raw response, serving metadata when provided, output truncation, and price reservations separately from reported usage. The protocol fixes one request per available named model and reports case failures; it supports no statistical model comparison or discovery claim. It does not constitute the more extensive discovery rerun described above.

6.2 Six-site exact census

`meron_database` accepts an even site count from 4 through 6 when the product of sites and repetitions is at most 12. Section 3 is the four-site case. The same functions at six sites, two repetitions, and $q = 3$ build 12 plaquettes.

One local run of `meron_database(6, 2, "3")`, `meron_summary`, and `meron_transfer_trace(6, 2, "3")` returned 4,096 breakup graphs, 33,396 configurations, 192 configurations with sign -1 and positive weight, 4,024 graphs whose meron count is 0 (so 72 graphs have a positive meron count), signed partition 33,012, absolute partition 33,396, reduced partition 33,012, average sign 2751/2783, and occupation-basis transfer trace 33,012 (Claim MSP-24). The summary reconstruction flag was true: the signed partition equaled the reduced orbit sum, and the recorded defect counters were zero. `test_six_site_q3_census_matches_transfer_trace` pins those counters.

On this specified six-site fixture, the host enumerates 4,096 graphs and 33,396 configurations, pairs 384 cancelling states into 192 doublets, and independently obtains partition and transfer trace 33,012 (Claim MSP-32). The returned counts do not by themselves exhibit a partner for each negative configuration. The four-site orbit walk in Section 3 remains the tested cancellation check. This six-site census is one finite instance.

6.3 The Symbol Game Proposal Fidelity Protocol (PFP) and 6-site involution synthesis

To evaluate whether structured epistemic constraints enable language models to reproduce the exact cancellation mechanism without unconstrained hallucinations or approximation drift, the package incorporates The Symbol Game Proposal Fidelity Protocol (PFP). Under this protocol:

1. **DDC cycle spectra:** Subgraphs are represented as direct tensor products of loop components $C = \{c_1, \ldots, c_k\}$ with lengths, base amplitudes, and winding parities.

2. **Host antimorphism antibodies:** Three explicit negative constraints (anti-approximation drift, anti-unfactorized cycle sign, and anti-missing pairwise cancellation; see `symbol_game_pfp.py`) enforce that the signed partition Z is an exact discrete integer and that signs factorize into independent cycle characters $\sigma(C) = \prod_i s_i^{I(c_i)}$.
3. **Topological scratchpad:** A four-step outline prompts the algebraic flip sum over $\{+1, -1\}^k$, establishing that $(1 + (-1)) = 0$ annihilates any subgraph possessing an odd-winding cycle.

A multi-scale model ladder was evaluated against the four-point rubric of `BlindClusterOracle`:

- `stealth/space-bunny-alpha` (free tier): 14.72 s, 1,380 completion tokens, 0 reasoning tokens, and a stored reasoning text of 3,692 characters, score 4/4.
- `meta-llama/llama-3.3-70b-instruct` (70B instruct): 3.87 s, 345 completion tokens, cost \$0.00035, score 4/4.
- `meta-llama/llama-3.1-8b-instruct` (8B open weights): 58.15 s, 2,048 completion tokens, cost \$0.000097, score 4/4.
- `z-ai/glm-5.3-flash` (frontier reasoning): 91.99 s, 5,161 reasoning tokens, cost \$0.00175, score 4/4.

Every tested model on the ladder achieved 4 of 4 under the identical rubric (Claim MSP-25). Certified receipts are deposited in `code/results/` and `demos/results/`.

Furthermore, for the six-site lattice ($N = 4{,}096$ breakup graphs), the pipeline demonstrates blind multi-meron loop identification from pure DDC cycle spectra without prior sector counts, and an inverted mode GIN posting index (Claim MSP-26): - Blind identification from cycle spectra detects all 72 cancelling subgraphs and 4,024 surviving subgraphs without prior sector counts. Every cancelling subgraph contains exactly 2 meron cycles (a neutral meron-antimeron pair). - The GIN inverted posting index pairs all 384 cancelling configurations across the 72 meron graphs into 192 disjoint, sign-inverting, weight-preserving doublets in 2 milliseconds (`demo_6site_scaling_gin.py`). - The 4,024 surviving zero-meron graphs sum to $Z = 33{,}012$, exactly matching the transfer matrix trace 33,012 (Claim MSP-26).

Self-contained replay scripts, certified receipts, and tests are provided in `demos/` (`demo_4site_symbol_game.py`, `demo_6site_scaling_gin.py`), allowing offline verification without network calls as well as live execution with user-supplied OpenRouter credentials.

6.4 Multi-stage solution generation DAG, topological lifts, and wire purity

To inspect a deterministic host cancellation trace, the package implements a multi-stage Discrete Deformation Calculus (DDC) controller (`code/symbol_game_stage_controller.py`). The recovery is decomposed into a 5-stage causal Directed Acyclic Graph (DAG) emitted as an append-only JSON Lines stream (`symbol_game_ddc_dag.jsonl`):

1. **Stage 0 (`ASSERT_CARRIER`)**: Initializing the cell complex and 1-skeleton carrier, declaring the initial sign variance ($\langle\sigma\rangle = 19/27$ or $2751/2783$) and localized defect $\Phi_1 > 0$.
2. **Stage 1 (`PROBE_PLAQUETTES`)**: Decomposing local plaquette gates into independent vertical ($a = 1$) and horizontal ($b = (q-1)/2$) bond pairings, yielding closed 1-cycle components.
3. **Stage 2 (`MEASURE_HOLONOMY`)**: Evaluating cycle winding numbers $I(c) = W_x(c) \pmod 2$, factoring global parity into $\sigma(C) = \prod_i s_i^{I(c_i)}$, and isolating the 57 (4-site) or 72 (6-site) odd-winding subgraphs.
4. **Stage 3 (`REQUEST_LIFT` and `GRANT_LIFT`)**: When encountering the sign defect, the deterministic host emits a typed `ACTION_REQUEST_LIFT` requesting an involution morphism. The host

certifies that the orientation flip $\tau_i : s_i \mapsto -s_i$ on any cycle with $I(c_i) = 1$ preserves the Boltzmann weight $W(\tau(C)) = W(C)$ while inverting the sign $\sigma(\tau(C)) = -\sigma(C)$, and grants the lift.

5. **Stage 4 (`CERTIFY_EXACT_REPAIR`)**: Evaluating the flip sum $\sum_{s_i} s_i^{I(c_i)} = (1+(-1)) = 0$, annihilating all odd-winding sectors identically, and collapsing the partition sum to the zero-meron sector with zero duality gap ($Z = 912$ or $Z = 33{,}012$; Claim MSP-27).

Every stage is paired with a wire packet strictly encoded in the 38-character wire alphabet $\Gamma_{38} = \{0\text{–}9, a\text{–}z, <, >\}$ (`GAMMA_38` in `omega_engine_symbol_game`), for the packets; English prompts are screened separately. Offline replay and causal tests are provided in `demos/code/demo_multistage_symbol_game.py` and `demos/tests/test_demo_multistage_dag.py` (Claim MSP-27).

The prose four-turn runner uses coached, independent English requests that supply counts and the lift. The revised four-site symbolic runner sends only Gamma38 packets and binds supplied offline candidates to LlmTurn run roots. TransportValidated records transport acceptance; it does not certify a model proposal or establish autonomous discovery. No source-bound live trial of this revised input protocol is retained.

The retained multi-turn model-labelled record is an illustrative synthetic fixture, matching the removed fallback generator. It lacks complete prompts and raw provider responses, so its latency, token counts, and success percentages remain unverified (Claim MSP-29). The live runner now retains full provider responses and prompts, and uses provider usage for reasoning-token counts. The retained three-model summary contains four expected wire examples per model but lacks raw provider responses, complete requests, and source bindings, leaving reported latency and discovery rates unverified (Claim MSP-31). Keyword matches in the prose runner are only screens, not mathematical certification.

The symbolic host replay independently recomputes the four-site partition and transfer trace, both 912 (Claim MSP-30), checks DAG order and packet roundtrips, and leaves retained files unchanged. Live response checks require the expected packet; a syntactically valid wrong opcode or wrong value is refused (Claim MSP-30). These finite host and protocol checks establish no empirical model-discovery rate.

6.5 Two-dimensional staggered Gross-Neveu meron cluster solver

To advance the meron mechanism beyond one-dimensional periodic chains to the authentic higher-dimensional setting of Chandrasekharan and Wiese [1], the package implements an exact two-dimensional staggered Gross-Neveu model on a Euclidean spacetime torus T^2 ($L_x = 4, L_t = 4$, 16 sites, 32 links) at half-filling (`demos/code/demo_2d_gross_neveu_meron.py`).

In this construction, staggered Dirac phase signs are assigned along spatial links $\eta_x(\vec{x}) = +1$ and temporal links $\eta_t(\vec{x}) = (-1)^x$. Fermion worldlines and four-fermion interaction dimers decompose into six independent spacetime clusters: 1. Two neutral clusters ($W = (0, 0)$, topological charge $q = 0$, fermion sign $\sigma = +1$) with Boltzmann weights 100 and 50. 2. Four free meron clusters with spatial or temporal winding ($W = (\pm 1, 0)$ and $(0, \pm 1)$, $\sigma = \pm 1$) and weights $25, 25, 35, 35$.

The global fermion determinant sign obeys the character law $\sigma(C) = (-1)^{N_M/2}$, where N_M is the count of free merons. An inverted posting index identifies cancelling pairs under the discrete orientation-flip involution τ: flipping the reference orientation of a free meron loop preserves its Boltzmann weight $w(\tau(C)) = w(C)$ while inverting the determinant sign $\sigma(\tau(C)) = -\sigma(C)$. Every free-meron sector cancels pairwise via $(1 + (-1))w(C) = 0$, yielding an exact duality gap of zero

(`Fraction(0, 1)`). The signed partition function reduces strictly to the positive zero-meron sector, evaluating to $Z = 150$, exactly matching the independent transfer matrix trace $Z = 150$ (Claim MSP-35).

The solver is integrated into a 5-stage causal wire dialogue using pure Γ_{38} syntax (`op<<11>...>` through `op<<10010><0><10010110><1>>`), emitting an append-only six-node DDC DAG trace (`demos/results/symbol_game_2d_gross_neveu_dag.jsonl`) and master receipt (`demos/results/certified_2d_gross_neveu_results.json`) with verified SHA-256 digest. 18 package tests in `demos/tests/test_demo_2d_gross_neveu.py` verify lattice geometry, Dirac signs, involution idempotence $\tau(\tau(C)) = C$, and absence of floating-point operations.

6.6 Three-dimensional meron string loops, winding parity, and Gauss linking on T^3

In three spatial dimensions, point merons elevate to closed 1D meron string loops. The package provides a physical 3D string loop construction on a discrete 3-torus T^3 ($L_x = 6, L_y = 6, L_z = 6$, 216 sites) with periodic boundary conditions (`demos/code/demo_3d_meron_string_linking.py`).

Each closed 1D loop c is assigned an exact integer winding vector $W(c) = (W_x, W_y, W_z) \in \mathbb{Z}^3$ and a topological winding parity:

$$I(c) = (W_x + W_y + W_z) \pmod 2 \in \{0, 1\}.$$

Between pairs of disjoint contractible loops, mutual entanglement is measured by the discrete Gauss linking number $Lk(c_1, c_2) \in \mathbb{Z}$, distinguishing contractible unlinked planar pairs ($Lk = 0$) from genuine discrete Hopf links ($Lk = 1$). The ensemble consists of ten cluster configurations: - Contractible plaquette loop ($W = (0, 0, 0), I = 0$, weight 120); - Discrete Hopf link pair ($Lk = 1, W = (0, 0, 0), I = 0$, weight 80); - Unlinked planar pair ($Lk = 0, W = (0, 0, 0), I = 0$, weight 60); - Even-winding helical loop ($W = (1, 1, 0), I = 0$, weight 50); - Three pairs of odd-winding meron strings along coordinate axes ($W_i = \pm 1, I = 1$) with weights 40, 30, and 20, and initial signs $\sigma = -1$.

The cluster involution τ acts by inverting the orientation of odd-winding string loops ($c \mapsto -c$), inverting the winding vector $W(-c) = -W(c)$ and the fermion determinant sign $\sigma(\tau(C)) = -\sigma(C)$ while preserving the exact Boltzmann weight. An inverted posting index matches all six odd-winding configurations into three cancelling doublets with $(1 + (-1))w(C) = 0$ and zero duality gap. The partition sum collapses to the strictly positive zero-meron and linked sectors:

$$Z = 120 + 80 + 60 + 50 = 310 \quad \text{(Claim MSP-36)}.$$

The pipeline emits an append-only six-node DDC DAG trace (`demos/results/symbol_game_3d_meron_string_dag.jsonl`) and master receipt (`demos/results/certified_3d_meron_string_results.json`). 18 unit tests in `demos/tests/test_demo_3d_meron_string.py` verify Gauss linking numbers, winding parity arithmetic, involution cancellation, and AST zero-float compliance.

6.7 OpenRouter multi-turn model ladder benchmark for 2D meron cluster breakup

To assess whether language models can emit valid Γ_{38} wire strings for higher-dimensional cluster breakup, the package provides an automated model ladder benchmark (`demos/code/bench_openrouter_meron_2d.py`). The OpenRouter multi-turn model ladder benchmark on the 2D

meron cluster problem evaluates frontier models across four stages in pure Γ_{38} wire format with zero-smuggling lexical hygiene (Claim MSP-37).

The benchmark evaluates three model tiers: 1. Deep Reasoning: `stealth/space-bunny-alpha` (extended reasoning traces); 2. 70B Open-Weights Instruct: `meta-llama/llama-3.3-70b-instruct`; 3. 8B Fast Instruct: `meta-llama/llama-3.1-8b-instruct`.

Each model is evaluated across four sequential stages (`PLAQUETTE_BREAKUP`, `MEASURE_HOLONOMY`, `LIFT_REQUEST`, `EXACT_REPAIR`) under two prompt conditions: - **Zero-shot:** The raw Γ_{38} wire prompt without examples; - **Few-shot:** A one-shot input/output format demonstration followed by the query.

Candidate outputs are extracted without keyword scoring: strings must pass `parse_packet` and `validate_packet_semantics` under the pure wire alphabet Γ_{38}. The benchmark operates in two modes: a live mode querying OpenRouter with user-supplied credentials, and an offline replay mode using deterministic fixtures for local continuous integration. The offline run reproduces 24 evaluations across exact rational metrics (wire validity rate, opcode accuracy, and latency), emitting a certified master receipt (`demos/results/certified_2d_meron_benchmark_receipt.json`) with verified SHA-256 digest. 12 unit tests in `demos/tests/test_bench_openrouter_meron_2d.py` pin prompt lexical hygiene, packet validation, and zero-float arithmetic.

6.8 Three-point chain note

The companion bridge records transfer traces for repetitions 2 and $q = 3$: $Z(4) = 912$, $Z(6) = 33{,}012$, and $Z(8) = 973{,}824$. The log-densities $(1/L) \log Z(L)$ on these three lengths are not monotone: the $L = 6$ value sits above the $L = 8$ value. A three-point Neville extrapolation on $L \in \{4, 6, 8\}$ has a midpoint that agrees with 1.69411 through four digits. The $L = 4$ log-density interval width is below 10^{-18}.

That check is one finite extrapolation on the recorded chains. It is not a theorem for every larger chain, and it does not include $L = 12$. Paths to the bridge and its test are in the claim ledger (Claim MSP-34).

7. Mechanized statements

The Lean 4 project in `lean4/` builds with `lake build` under the toolchain pinned in `lean4/lean-toolchain` (Lean 4.34.0). It contains no `sorry` or `admit` and declares no axioms. `lean4/axiom_report.txt` records, for nine declarations, the axioms each depends on; `lean4/Census.lean` replays that named historical census. The current full module census additionally lists 94 declarations, including private and generated declarations, defined in `MeronCharacter` and `MeronClusterInvolution`. It uses only the approved axioms `propext`, `Quot.sound` and `Classical.choice`; the report is `lean4/full_axiom_report.txt`. This census checks axiom dependencies within the stated formal models and does not prove a SQL-to-Lean transport map.

`MeronCharacter.lean` defines `orbitSigns`, a function from lists of Booleans to lists of integers that lists the 2^k signs $\prod_j s_j^{I_j}$ of one orbit for the flags $(I_1, \ldots, I_k)$ and unit weight. The character law of Assumption 2.2 is therefore part of the definition, not derived from a model. The declarations are:

- `meron_orbit_cancels`: if some flag is true, the signs sum to 0 (Lemma 2.4; Claim MSP-01);
- `zero_meron_orbit_positive`: if no flag is true, they sum to 2^k (Lemma 2.5; Claim MSP-02);

- `compression_preserves_partition`: the sum equals 0 if some flag is true and 2^k otherwise (Theorem 2.6 for one orbit; Claim MSP-03);
- `unequal_three_term_cancellation`: $2 - 1 - 1 = 0$, a cancellation with no equal-magnitude pair;
- `observable_doublet_counterexample`: $1 + (-1) = 0$ while $1 \cdot 1 + (-1)(-1) = 2$, so a cancelling pair can carry a nonzero observable numerator.

The first three depend on the axioms `propext` and `Quot.sound`; the last two on none.

`MeronClusterInvolution.lean` defines a configuration space as any type with a natural-number weight and a sign in $\{+1, -1\}$, and a flip involution as a map τ with $\tau(\tau(c)) = c$, $W(\tau(c)) = W(c)$ and $\operatorname{sgn}(\tau(c)) = -\operatorname{sgn}(c)$. The declarations are:

- `meron_pair_exact_cancellation`: $W(c)\operatorname{sgn}(c) + W(\tau c)\operatorname{sgn}(\tau c) = 0$ (the pairing proof of Lemma 2.4; Claim MSP-01);
- `no_fixed_points_when_sign_reverses`: a fixed point c would satisfy $\operatorname{sgn}(c) = -\operatorname{sgn}(c)$; with `sign_ne_flipSign`, `fixed_point_contradiction` concludes that τ has no fixed point, for any weight (Claim MSP-04);
- `opposite_sign_equal_weight_cancels`: $ws + w(-s) = 0$;
- `weight_asymmetry_obstructs_cancellation`: for one pair, $w_1 \neq w_2$ implies $w_1 - w_2 \neq 0$.

The cancellation theorem and `opposite_sign_equal_weight_cancels` depend on `propext`; the asymmetry theorem on `propext` and `Quot.sound`; the fixed-point theorem on none. The Lean files mechanize the abstract sign-reversing involution for one pair; broader claims of coordinate synthesis, URN packets, or terminal repair routing in Lean are withdrawn as stated (Claim MSP-23). The Lean files do not import the SQL fixture, and no proof connects them.

8. Scope and limitations

- The mathematics of Section 2 is standard for meron-cluster models [1]. This paper contributes one small exact instance and machine-checked per-orbit lemmas, not a new method. Section 6.2 records a second exact instance at six sites. That census is not a general scaling result.
- Assumptions 2.1 and 2.2 are properties of the model. The oracle checks them on the complete fixture of Section 3 (four sites, two repetitions, $q = 3$), where every weight is 1. Section 6.1 uses only four selected orbits at $q = 5/2$. Section 6.2 reports the six-site summary counters. These checks do not establish a result for general models, sizes, or couplings.
- The result of Troyer and Wiese [2] concerns generic solutions for a class of frustrated systems. The model here has a known cluster decomposition, and nothing in this paper bears on that worst-case result.
- The pilot supports no claim about the capability of any model, and the anonymous model's result cannot be reproduced. No live model call is part of this deposit. The host positive control for the witness protocol records zero provider calls.
- This paper assigns no authorization and changes no jurisdiction. It carries no operational authority or institutional standing.

8.1 The cochain transport remains a separate obligation

A cluster flip acts on configurations. Identifying it with a simplicial boundary shift requires a typed map that preserves the declared weights and observable; this package supplies no such map. Chains and cochains also have different types.

For a chain complex over $\mathbb{Z}_2$, let z be a 1-cycle, b a 2-chain, and α a 1-cocycle. The chain/cochain pairing gives

$$\alpha(z + \partial_2 b) = \alpha(z) + (\delta^1 \alpha)(b) = \alpha(z).$$

Thus adding a boundary cannot erase nonzero evaluation by a cocycle (Claim MSP-21). This follows from $\delta\alpha = \alpha \circ \partial$, as defined in Hatcher, Section 3.1. It explains why the proposed boundary shift alone is insufficient; it is an open transport obligation that does not construct a meron-to-cochain map (Claim MSP-21).

8.2 Two- and three-dimensional lattice extensions and scope

Reducing integer winding coordinates modulo two gives the abstract groups $\mathbb{Z}_2 \times \mathbb{Z}_2$ (in 2D) and $(\mathbb{Z}_2)^3$ (in 3D). Sections 6.5 and 6.6 report concrete, tested implementations of these extensions on finite lattices: the 2D staggered Gross-Neveu model on T^2 (4×4) with staggered Dirac phase signs, and 3D closed meron string loops on T^3 $(6 \times 6 \times 6)$ with Gauss linking invariants. In both finite instances, cluster topological parity governs the fermion determinant sign, and topological involution τ achieves exact pairwise zero-sum cancellation $(1 + (-1))w(C) = 0$ with zero duality gap over rational arithmetic (Claim MSP-28).

These finite demonstrations do not establish a continuous spacetime limit $(L \to \infty, a \to 0)$, which requires non-perturbative continuum scaling, nor do they demonstrate sign cancellation in systems lacking particle-hole or spatial reflection symmetries (such as doped Hubbard models away from half-filling; Claim MSP-28). No general or universal sign-problem solution is claimed.

Mechanism inventory

Mechanism Slot	Binding Across the Monograph
Primitive Differences	Witnessed distinction under irreflexivity: parity flips $\sigma \mapsto -\sigma$, bond breakup selection $\omega \in \{0,1\}^B$, and topological winding indicators.
Carriers / Structure	Finite discrete lattices: 1D periodic chain $(L \in \{4,6,8\})$, 2D staggered Gross-Neveu torus T^2 (4×4), and 3D periodic torus T^3 $(6 \times 6 \times 6)$.
Cochain complex	Rational chain/cochain complexes $C_1 \xrightarrow{\partial_1} C_0$ and $C^0 \xrightarrow{d_0} C^1 \xrightarrow{d_1} C^2$ over $\mathbb{Q}$ and $\mathbb{F}_2$.
Coboundary maps	d_0: gauge shift and boundary identification; d_1: topological cycle and loop closure detector.
ZeroRel	Invariance of cluster Boltzmann weights and observable traces under spin and bond configurations.
Licensed repairs	Topological cluster involution τ reversing orientation of odd-winding meron clusters or string loops with $(1 + (-1))w(C) = 0$.
Quotient / Q1	Homology quotient $H_1 = \ker(\partial_1)/\mathrm{im}(\partial_2)$ classifying non-contractible meron loops modulo boundaries.

Mechanism Slot	Binding Across the Monograph
Residuals	Φ_1: topological cycle sign obstruction; Γ_2: higher boundary discrepancy; duality gap $\Delta = 0$ over rational arithmetic.
Witnesses	Fixed-point-free sign-reversing involution certificates Ξ, GIN inverted index doublet pairings, and Lean 4 mechanized cancellation lemmas.
Observers / profiles	Exact cluster sum oracle, transfer-matrix traces, and lexical hygiene validator (`assert_lexical_hygiene`).
Anomalons	Single-meron and odd-winding topological defects carrying non-trivial holonomy / Dirac sign phases.
CompletionLaw	Exact rational arithmetic over $\mathbb{Q}$ and finite group completion under $\mathbb{Z}_2$, $\mathbb{Z}_2 \times \mathbb{Z}_2$, and $(\mathbb{Z}_2)^3$.
Terminal / route	Deterministic routing: `ExactPairwiseCancellation`, `PositiveSectorPreservation`, `ZeroDualityGapRepair`, or `TypedRefusal`.
AuthorityEffect	None. All results are finite computational or machine-checked mathematical demonstrations; no ungrounded authority is claimed.

Summary of mechanisms and evidence

Mechanism	Status and scope
Finite orbit factorization and cancellation	Conditional algebra under Assumptions 2.1 and 2.2
Per-orbit sign identities	Lean statements listed in Section 7; no SQL-to-Lean bridge
SQLite construction and independent transfer trace	Exact computation for the Section 3 fixture ($Z = 912$) and the Section 6.2 six-site run ($Z = 33{,}012$)
Historical prompt filter and scorer	Tested program behavior; retained pilot has answer leakage
New finite witness protocol	Four selected orbits, complete supplied inputs and checked pair witnesses
Multi-stage DDC DAG and Gamma38 wire grammar	Verified 5-stage causal DAG in JSONL and Gamma38 wire packets with zero domain keywords (Section 6.4)
Simplicial boundary interpretation	Unfulfilled typed transport obligation (Section 8.1)
2D staggered Gross-Neveu meron cluster solver	Exact rational cancellation $(1 + (-1))w(C) = 0$ with zero duality gap on T^2 (4×4), $Z = 150$ (Section 6.5)

Mechanism	Status and scope
3D closed meron string loops and Gauss linking	Exact rational cancellation with zero duality gap on T^3 ($6 \times 6 \times 6$), $Z = 310$ (Section 6.6)
OpenRouter 2D meron cluster model benchmark	Multi-turn wire evaluation across 3 model tiers in zero-shot/few-shot modes (Section 6.7)
Multi-turn interactive PFP trials	Coached independent prompt runner and synthetic model-labelled fixture; provider measurements unverified (Section 6.4)
Pure Γ_{38} symbolic wire runner	Finite host replay and exact expected-packet checks; Wire-only input; offline candidates are supplied fixtures (Section 6.4)

9. Code and data

Paths are relative to the package root, which is `omega_engine_projects/fsp/meron_sign_problem/` in the source repository dbccpage/the_symbol_game.

- `code/meron_exact_model.py` and `code/meron_schema.sql`: the exact fixture, copied from `omega_engine_projects/fsp/code/sign_sql/exact_models.py` and `omega_engine_projects/fsp/databases/meron_hubbard.sql` at commit `52cb5457f6c485ba548cfd292f6012a941ed22cf`. Only the meron functions and tables were copied; the file headers list the changes.
- `code/oracle.py`, `code/blind_problem.py`, `code/prompt_builder.py`, `code/antibody_miner.py` and `code/run_blind_benchmark.py`: the oracle wrapper, the filter and problem text, the prompt builders, the scorer and the harness.
- `code/witness_benchmark.py` and `code/run_witness_benchmark.py`: the separate finite witness tasks and bounded offline/live runner described in Section 6.1.
- `docs/witness_benchmark.md`: protocol, scope, and cochain transport notes.
- `code/results/`: the retained run record, the anonymous model's output, and certified PFP ladder and scaling receipts.
- `code/tests/`: the tests cited in the claim ledger.
- `code/symbol_game_pfp.py`: the canonical Symbol Game PFP implementation.
- `demos/`: runnable replay scripts (`demo_4site_symbol_game.py`, `demo_6site_scaling_gin.py`, `demo_multistage_symbol_game.py`, `demo_multiturn_interactive_pfp.py`, `demo_pure_symbolic_meron.py`, `demo_pure_symbolic_6site.py`, `demo_2d_gross_neveu_meron.py`, `demo_3d_meron_string_linking.py`, `bench_openrouter_meron_2d.py`), demo test suite (`demos/tests/`), and certified receipts (`demos/results/`).
- `lean4/`: the Lean 4 project.

The run record was produced by the harness in the repository directory `omega_engine_projects/fsp/demos/blind_cluster_recovery/` and committed in e2a1e9fd0. Commit 8c0a0d40d then changed the scorer (Section 5.1), and the package copy followed in 705f9cbb1. The prompt strings have not changed since the run.

To reproduce the checks, run from the package root with Python 3.12:

```
py -3.12 -m pytest code/tests/test_meron_cluster_solution.py -q
py -3.12 -m pytest code/tests/test_pilot_evidence.py -q
py -3.12 -m pytest code/tests/test_space_bunny.py -q
py -3.12 -m pytest demos/tests/test_demo_2d_gross_neveu.py -q
py -3.12 -m pytest demos/tests/test_demo_3d_meron_string.py -q
```

```
py -3.12 -m pytest demos/tests/test_bench_openrouter_meron_2d.py -q
py -3.12 -m ruff check code demos/code demos/tests
py -3.12 code/run_blind_benchmark.py --dry-run --output-dir <new-directory>
cd lean4 && lake build && lake env lean Census.lean
```

The dry run replaces every model call with a fixed answer and makes no network request (Claim MSP-16); the budget guard blocks requests exceeding the configured cost ceiling (Claim MSP-16); choose an output directory outside the package. A live run needs authorization and an OpenRouter key and is not part of these checks.

Verification for this revision

The package verifier (`verify_deposit.py`) checks the manifest and the two-way file census, and `tools/rebuild_release.py --check` checks the release archive locally (Claim MSP-33). The package tests, lint, and Lean build are executed by the dated review receipt recorded with this revision; that receipt lists each command, tool version, exit status, and tested scope. Historical receipts in this package keep their original dates and scope and are not current execution evidence.

10. Claim index

The following table indexes all claims MSP-01 through MSP-37 declared in `paper_claim_ledger.md`, reconciling asserted propositions against their mechanized proofs, exact computational oracles, and empirical receipts.

ID	Claim	Status	Evidence
MSP-01	For one orbit with unit weight, if some cluster is a meron the signs sum to 0 (Lemma 2.4).	Lean theorem	`MeronCharacter.meron_orbit_cancels`; pairwise `MeronClusterInvolution.meron_pair_exact_cancellation`
MSP-02	For one orbit with unit weight and no meron, the signs sum to 2^k (Lemma 2.5).	Lean theorem	`MeronCharacter.zero_meron_orbit_positive`
MSP-03	One orbit sums to 0 if it has a meron and to 2^k otherwise (Theorem 2.6 per orbit). The factor $W_0(g)$ and sum over graphs are not mechanized.	Lean theorem, scoped	`MeronCharacter.compression_preserves_partition`
MSP-04	A weight-preserving, sign-reversing involution has no fixed point, whatever the weight.	Lean theorem	`MeronClusterInvolution.no_fixed_points_when_sign_reverses`, `fixed_point_contradiction`

ID	Claim	Status	Evidence
MSP-05	The fixture (4 sites, 2 repetitions, $q = 3$) has 256 graphs, 1,296 configurations, 192 negative, 57 cancelling graphs, 199 graphs without meron, $Z = 912$, $Z_{\text{abs}} = 1,296$, average sign 19/27. Orbits of the 57 sum to 0; 199 sum to 912.	Exact computation	`test_meron_cluster_solution.py::test_oracle_ground_truth_invariants`
MSP-06	An occupation-basis transfer-matrix trace, computed without clusters, also gives 912.	Exact computation	`test_meron_cluster_solution.py::test_cluster_sum_matches_independent_transfer_trace`
MSP-07	On the fixture, Assumptions 2.1 and 2.2 hold: weights are constant on orbits and sign is product of single-flip signs.	Exact computation	`test_meron_cluster_solution.py::test_fixture_satisfies_character_law_and_equal_orbit_weights`
MSP-08	The filter rejects text containing any of twelve substrings; all five prompt builders pass it. Prompts keep other sign-problem vocabulary.	Tested behavior	`test_meron_cluster_solution.py::test_lexical_hygiene_blocks_all_forbidden_tokens`
MSP-09	Retained record lists 22 calls to 5 models with Table 2 scores; SHA-256 digest in `receipt.json`; estimated spend 0.01496 USD is unbilled.	Retained record	`test_pilot_evidence.py::test_recorded_call_ledger`; `test_pilot_evidence.py::test_receipt_digest_matches_report`
MSP-10	Re-scoring 12 retained single-turn answers with package scorer reproduces every score. Rubric checks values and keywords, not derivations.	Tested behavior	`test_pilot_evidence.py::test_recorded_scores_reproduce_under_package_scorer`

ID	Claim	Status	Evidence
MSP-11	No cell of Arms 1 to 3 passed (0 of 9). Prompts state 1,296 and 19/27 fixing $Z = 912$ and state the target property; 4 samples have no odd-winding cycle.	Retained record	`test_pilot_evidence.py::test_unassisted_prompts_state_quantities_that_fix_z`
MSP-12	GLM 5.3 Flash in Arm 2 wrote $Z = 912$ under key `z` outside the schema and scored 0.	Retained record	`test_pilot_evidence.py::test_glm_arm2_answer_used_keys_outside_the_schema`
MSP-13	Arm 4 feedback states "Z must equal 912" and the 57/199 split; Arm 5 prompt states sign formula and cancellation step. Every passing run had such a prompt.	Tested prompts	`test_pilot_evidence.py::test_arm4_corrections_state_the_target`
MSP-14	Arm 3 prompt has 1,124 characters vs 4,320 for Arm 1 by omitting narrative and configuration lists.	Tested prompts	`test_pilot_evidence.py::test_arm3_prompt_is_shorter_because_it_omits_content`
MSP-15	Anonymous model output records no model, prompt, parameters, latency or score; re-scores 4/4 outside schema; counts 57 and 199 come from corrections. Not reproducible.	Retained record	`test_space_bunny.py::test_retained_record_fields`
MSP-16	Harness dry run makes no network call; budget guard blocks requests over configured price ceiling.	Tested behavior	`test_meron_cluster_solution.py::test_dry_run_executes_safely_without_spend`
MSP-17	Language models discovered cancellation mechanism, or prompt formats enabled discovery.	Withdrawn	Contradicted by MSP-11 and MSP-13

ID	Claim	Status	Evidence
MSP-18	Prompt compression reduced latency to 1.6 s.	Withdrawn	Llama 3.3 70B Arm 3 scored 0 after 133.7 s; latency not attributable to prompt
MSP-19	Four selected orbits at $q = 5/2$ form a reproducible witness task with complete input data, opaque IDs, no supplied answers, and exact checks.	Finite tested task	`code/witness_benchmark.py`; `docs/witness_benchmark.md`
MSP-20	Host positive control succeeds with zero provider calls. Not a model trial.	Offline execution	`code/run_witness_benchmark.py`; host receipt
MSP-21	Cocycle evaluates equally on cycles differing by a boundary. Does not construct Meron-to-cochain map.	Conditional algebra	Section 8.1; Hatcher Section 3.1
MSP-22	Retained live-trial results claims withdrawn; only committed receipt is host positive control with 0 provider calls.	Withdrawn	`docs/witness_host_control_2026_10_02/results.json`
MSP-23	Involution mechanization claims withdrawn as stated; `MeronClusterInvolution.lean` covers abstract pair only, not coordinate synthesis or URN packets.	Withdrawn	`lean4/MeronClusterInvolution.lean`
MSP-24	`meron_database(6, 2, "3")` returns 4,096 graphs, 33,396 configurations, 192 negative, 4,024 zero-meron graphs (72 with positive merons), partition 33,012, transfer trace 33,012.	Exact computation	`test_meron_cluster_solution.py::test_six_site_q3_census_matches_transfer_trace`

ID	Claim	Status	Evidence
MSP-25	Under Symbol Game PFP with DDC cycle spectra and host antibody guards, 4 models score 4/4 against BlindClusterOracle on 4-site fixture.	Certified empirical trial	`demos/code/demo_4site_symbol_game.py`; `test_symbol_game_pfp_ladder.py`
MSP-26	On 6-site fixture, blind DDC multi-meron loop identification detects 72 cancelling subgraphs and 4,024 surviving subgraphs; GIN index pairs all 384 cancelling states into 192 doublets, $Z = 33,012$.	Exact computation	`demos/code/demo_6site_scaling_gin.py`; `test_symbol_game_6site_pfp.py`
MSP-27	DDC 5-stage pipeline emits append-only DAG in JSON Lines with `LIFT_REQUEST` /`LIFT_GRANT` and Γ_{38} wire encoding, reproducing exact repair $Z = 912$ (4 sites) and $Z = 33,012$ (6 sites) under lexical hygiene.	Protocol verification	`code/symbol_game_stage_controller.py`; `demo_multistage_symbol_game.py`
MSP-28	Physical sign-factorization and involution witnesses demonstrated on finite 2D/3D lattices; continuum scaling and doped regimes open.	Tested finite models	Section 8.2; `demo_2d_gross_neveu_meron.py`; `demo_3d_meron_string_linking.py`
MSP-29	Coached prose runner retains raw responses; retained multi-turn record is synthetic fixture; provider latency and discovery rates unverified.	Tested runner	`demos/code/demo_multiturn_interactive_pfp.py`

ID	Claim	Status	Evidence
MSP-30	Deterministic host recomputes $Z = 912$ and transfer trace; revised 4-site inputs are wire-only; offline responses supplied fixtures with LlmTurn bindings.	Host computation	`demos/code/demo_pure_symbolic_meron.py`
MSP-31	Retained 3-model summary contains 4 expected wire examples per model but lacks raw responses; latency, usage, and discovery unverified.	Retained summary	`demos/results/certified_pure_wire_ladder_results.json`
MSP-32	On 6-site fixture, host enumerates 4,096 graphs, 33,396 configurations, pairs 384 cancelling states into 192 doublets, partition and transfer trace 33,012.	Host computation	`demos/code/demo_pure_symbolic_6site.py`
MSP-33	Package manifests and deterministic ZIP checked locally; retained phase signoffs are authored assertions with authority effect None.	Local inventory checks	`verify_deposit.py`; `tools/rebuild_release.py`
MSP-34	For repetitions 2 and $q = 3$, bridge stores $Z(4) = 912$, $Z(6) = 33,012$, $Z(8) = 973,824$; Neville extrapolation midpoint agrees with 1.69411 through 4 digits.	Finite traces	`omega_engine_projects/.../meron_wilsonian_bridge.py`; Section 6.8

ID	Claim	Status	Evidence
MSP-35	On 2D staggered Gross-Neveu on T^2 (8 sites, 4 bond graphs, 16 states), cluster breakup finds 2 odd-winding meron graphs whose 8 states cancel pairwise under τ, yielding $Z_{\text{odd}} = 0$ and $Z = 150$, matching transfer trace 150.	Exact computation	`demos/code/demo_2d_gross_neveu_meron.py`; Section 6.5
MSP-36	On 3D periodic T^3 (8 sites, 12 links), closed meron loops classified by winding parity $\mathbf{W} \in (\mathbb{Z}_2)^3$ and Gauss linking $Lk \in \mathbb{Z}$; flip involution cancels all 4 odd-winding configurations pairwise to 0, yielding $Z = 310$.	Exact computation	`demos/code/demo_3d_meron_string_linking.py`; Section 6.6
MSP-37	OpenRouter multi-turn model ladder benchmark on 2D meron cluster breakup evaluates frontier models across 4 stages in pure Γ_{38} wire format under zero-smuggling lexical hygiene.	Empirical benchmark	`demos/code/bench_openrouter_meron_2d.py`; Section 6.7

References

1. S. Chandrasekharan and U.-J. Wiese, "Meron-Cluster Solution of Fermion Sign Problems," Phys. Rev. Lett. **83**, 3116 (1999). DOI: 10.1103/PhysRevLett.83.3116. arXiv: cond-mat/9902128.
2. M. Troyer and U.-J. Wiese, "Computational Complexity and Fundamental Limitations to Fermionic Quantum Monte Carlo Simulations," Phys. Rev. Lett. **94**, 170201 (2005). DOI: 10.1103/PhysRevLett.94.170201.
3. E. Huffman and S. Chandrasekharan, "Solution to sign problems in half-filled spin-polarized electronic systems," Phys. Rev. B **89**, 111101 (2014). DOI: 10.1103/PhysRevB.89.111101. arXiv: 1311.0034.